

Project 4

contribution:

- 仓库: [Teddy-Yangjiale/opencv: Open Source Computer Vision Library](#)
- 查询链接: <https://github.com/opencv/opencv/pulls?q=is%3Apr+is%3Aclosed+author%3ATeddy-Yangjiale>
- PR 总数: 7 个 closed PR
- 合并情况: 6 个已合并, 1 个关闭未合并

总览

时间	PR 标题	状态	类型	平台 / 架构
2026-04-27	#28891 calib3d: unify behavior of findHomography and estimateAffine2d	关闭未合并	行为一致性修复	通用平台, <code>calib3d</code>
2026-04-28	#28894 videoio: fix C4576 build error with older ffmpeg versions	已合并	编译修复	Windows / MSVC / older FFmpeg
2026-04-30	#28914 videoio(dshow): fix crash in videoInput::start() when pVih is NULL	已合并	崩溃修复	Windows / DirectShow
2026-05-11	#28999 dnn: vectorize SigmoidFunctor using universal intrinsics	已合并	SIMD 优化	RISC-V RVV 1.0, 也适用于 Universal Intrinsics 支持平台
2026-05-14	#29033 rvv: use vlseg/vsseg instead of vlse/vsse in universal intrinsic interleave ops	已合并	RVV 内存访问优化	RISC-V RVV scalable HAL

时间	PR 标题	状态	类型	平台 / 架构
2026-05-18	#29057 Rvv core norm	已合并	RVV 正确性修复	RISC-V RVV 1.0
2026-05-18	#29058 RISC-V: add RVV convertScale paths for selected depth conversions	已合并	RVV 性能优化	RISC-V RVV 1.0

#28891: calib3d 行为一致性修复

- PR: <https://github.com/opencv/opencv/pull/28891>
- 创建时间: 2026-04-27
- 状态: 关闭未合并
- 目标分支: `4.x`
- 关联问题: [#22746](#)

平台与架构

该 PR 修改的是 `calib3d` 模块的公共 API 行为，不绑定特定 CPU 架构或操作系统。影响所有调用 `cv::findHomography()` 的平台。

修改文件

- `modules/calib3d/src/fundam.cpp`
- `modules/calib3d/test/test_homography.cpp`

修改前后代码对比

修改前，当点对数量少于 4 时，`findHomography()` 直接抛出异常：

```
1 if( npoints < 4 )
2     CV_Error(Error::StsVecLengthErr,
3         "The input arrays should have at least 4
corresponding point sets to calculate Homography");
```

修改后，返回空矩阵，使行为接近 `estimateAffine2d()`：

```
1 if( npoints < 4 )
2     return Mat();
```

同时新增回归测试，验证少于 4 个点时不抛异常并返回空矩阵：

```
1 EXPECT_NO_THROW(H = cv::findHomography(src, dst));
2 EXPECT_TRUE(H.empty());
```

修复内容

该 PR 试图解决 `findHomography()` 与 `estimateAffine2d()` 在输入不足时行为不一致的问题。原行为会抛出 `CV_Error`，对连续数据管线不够友好；新行为允许调用者通过 `.empty()` 判断失败，而不是必须捕获异常。

结果

PR 最终关闭但未合并。管理员评论：Discussed on the core team meeting. `estimateAffine2D` and derivatives should throw exception (assert) as array with less than 4 points is invalid input.

感想

这个 pr 是我给 opencv 的第一个 pr，第一次拉取仓库，创建分支，定位代码，跑通编译，最后提交 pr。我认为最大的难点在于编译。在我的 WSL2 环境下进行 CMake 配置时，遇到了链接器路径的问题。由于 WSL 自动继承了 Windows 的环境变量，CMake 错误地定位并试图链接我的 D 盘 Anaconda 目录下的 Windows 版本 `.lib` 库（如 TIFF 和 OpenJPEG），导致 `dangerous relocation` 编译报错。

通过清理编译缓存，并在 CMake 命令中显式添加 `-DWITH_TIFF=OFF -DWITH_OPENJPEG=OFF` 等参数禁用不相关的图像解码模块，成功绕开了环境冲突。

事实上，这个issue比我想的复杂，这是涉及opencv的规则问题。管理员最后的讨论得出"All similar functions should throw exception (CV_Assert). Insufficient amount of points is invalid input."这远不是我能想到或者借助工具得到的结论。虽然这个pr并没有被merged，但是让我成功走通了一遍流程。

#28894: older FFmpeg + MSVC C++17 编译错误修复

- PR: <https://github.com/opencv/opencv/pull/28894>
- 创建时间: 2026-04-28
- 合并时间: 2026-04-28
- 状态: 已合并
- 目标分支: 4.x
- 变更规模: 1 个文件, 新增 2 行, 删除 3 行
- 关联问题: #28875

平台与架构

- 平台: Windows
- 编译器: MSVC
- 语言标准: 严格 C++17 模式
- 依赖: older FFmpeg, 例如 FFmpeg 4.4.4

修改文件

- `modules/videoio/src/cap_ffmpeg_impl.hpp`

修改前后代码对比

修改前使用 FFmpeg 宏 `AV_TIME_BASE_Q`:

```
1 return av_rescale_q(ts, tb, AV_TIME_BASE_Q);
2 return av_rescale_q(ts_avtb, AV_TIME_BASE_Q, tb);
```

修改后改用 FFmpeg 标准函数 `av_make_q(1, AV_TIME_BASE)`:

```
1 return av_rescale_q(ts, tb, av_make_q(1, AV_TIME_BASE));
2 return av_rescale_q(ts_avtb, av_make_q(1, AV_TIME_BASE), tb);
```

修复内容

older Ffmpeg 头文件中, `AV_TIME_BASE_Q` 可能展开为 C99 compound literal:

```
1 (AVRational){1, AV_TIME_BASE}
```

该语法不是标准 C++, MSVC 在 `/std:c++17` 下会报 `error C4576`。用 `av_make_q(1, AV_TIME_BASE)` 替代后, 避免了 C99 compound literal, 从而修复 Windows/MSVC 构建失败。

感想

这个issue是有关编译兼容性的问题。最大的难点在于配环境和跑通编译。为此我在电脑上使用Visual Studio配置了MSVC编译器和相关库。依旧遇到了一些路径和依赖的问题, 不过现在有了ai工具, 我们能更加明白的看懂报错而不是无从下手。

Ffmpeg: AVRational关于修改前后的语法改动, 在ffmpeg官网上能清晰的查询到函数的最新写法, 所以在正确性方面可以保证。

#28914: DirectShow pvih == NULL 崩溃修复

- PR: <https://github.com/opencv/opencv/pull/28914>
- 创建时间: 2026-04-30
- 合并时间: 2026-05-05
- 状态: 已合并
- 目标分支: `4.x`
- 变更规模: 1 个文件, 新增 7 行, 删除 1 行
- 关联问题: [#28904](#)

平台与架构

- 平台: Windows
- 后端: DirectShow
- 设备场景: legacy camera 或 virtual camera, 例如 Microsoft DirectShow Ball Sample
- 架构: 不绑定具体 CPU 架构, 属于 Windows videoio 后端稳定性问题

修改文件

- `modules/videoio/src/cap_dshow.cpp`

修改前后代码对比

修改前直接断言 `pvih` 非空:

```
1 VIDEOINFOHEADER *pvih = reinterpret_cast<VIDEOINFOHEADER*>(VD-  
  >pAmMediaType->pbFormat);  
2 CV_Assert(pvih);
```

修改后增加空指针检查, 失败时返回 `false`:

```
1 VIDEOINFOHEADER *pvih = reinterpret_cast<VIDEOINFOHEADER*>(VD-  
  >pAmMediaType->pbFormat);  
2 if (pvih == NULL) {  
3     DebugPrintOut("ERROR: pbFormat field is not set!\n");  
4     return false;  
5 }
```

修复内容

某些 DirectShow 设备或虚拟摄像头会出现特殊情况:

- `GetConnectedMediaType(&mt)` 返回 `S_OK`。
- 但 `mt.pbFormat` 仍然是 `NULL`。
- 原代码 `CV_Assert(pvih)` 会导致进程中止。

修改后不再让应用直接崩溃，而是让 `videoInput::start()` 返回失败，使上层 `cv::VideoCapture::open()` 或 `isOpened()` 能正常表达打开失败。

感想

这个pr中我学会了使用debug模式，我先下载了摄像头驱动，然后在运行时一步一步分析有关变量，最后找出的这个问题。这让我对于CV_Assert, CV_Error和return false这几种处理错误的方式有了更深的了解。

[modules/core/doc/intro.markdown](#)在opencv的这个文件中，有一段是关于Error Handling的，我最大的收获是通过这个pr，搞清了opencv中处理错误的方式，也体会到了参数检查的重要性，终于体会了project1布置的良苦用心。

#28999: DNN SigmoidFunctor Universal Intrinsic 向量化

- PR: <https://github.com/opencv/opencv/pull/28999>
- 创建时间: 2026-05-11
- 合并时间: 2026-05-12
- 状态: 已合并
- 目标分支: `4.x`
- 变更规模: 1 个文件, 新增 49 行

平台与架构

- 实测平台: RISC-V RVV 1.0, SpacemiT K1 SoC
- OpenCV 路径: `dnn` 模块
- 技术基础: OpenCV Universal Intrinsic
- 架构影响: 主要收益在支持 SIMD / scalable SIMD 的平台; PR 验证集中在 RVV

修改文件

- `modules/dnn/src/layers/elementwise_layers.cpp`

修改前后代码对比

修改前, `SigmoidFuncutor` 只有标量计算函数, 批量 `apply()` 走默认标量循环:

```
1 float calculate(float x) const
2 {
3     float y = 1.f / (1.f + exp(-x));
4     return y;
5 }
```

修改后新增向量化 `apply()`, 使用 Universal Intrinsics 处理 `float32` 数据:

```
1 v_float32 one = vx_setall_f32(1.0f);
2 v_float32 min_val = vx_setall_f32(-80.0f);
3 v_float32 max_val = vx_setall_f32(88.0f);
4
5 x = v_min(v_max(x, min_val), max_val);
6 vx_store(dstptr + i, v_sub(one, v_div(one, v_add(one,
    v_exp(x)))));
```

核心计算从标量:

```
1 1 / (1 + exp(-x))
```

改成适合向量化的等价形式:

```
1 1 - 1 / (1 + exp(x))
```

优化内容

该 PR 为 DNN 中的 `SigmoidFuncutor` 增加向量化批处理实现, 主要优化点包括:

- 使用 Universal Intrinsics, 让 SIMD 平台可以一次处理多个 `float`。
- 使用 `[-80.0f, 88.0f]` clamp, 避免 `v_exp` 溢出和极小值导致的异常慢路径。
- 使用 2-way unrolled loop, 降低 `v_exp` 延迟对吞吐的影响。
- 保留尾部 scalar fallback, 保证任意长度输入都能正确处理。

感想

这个pr中，函数的实现不难。这是一个dnn的激活层函数的通用内联的优化实现。最难的是配置开发板。我们拿到一块空的开发板，目标是烧录操作系统进去。我是买的香蕉派 BananaPi-F3，8核，支持RVV1.0。我一开始按照官网的文档，将bianbu的镜像烧录到emmc中，但是失败了。在这里我遇到了整个月最艰难的挑战。因为关于这块开发板全网只有一个简略的官方文档和官方的一个知乎讲解。并不足以解决烧录失败的问题，ai解决不了，论坛也没有回复，商家也是代理的厂家，也无法解决，问学长或老师也没办法解决。好在是后来新买了TF卡，将镜像烧到卡里才得以成功启动。后续就是正常配置ssh，拉取代码，交叉编译。

回过头来看，感觉最难的是没有足够的信息的时候，怎么切换思路并且反复尝试。实验室的学长都是直接ssh就可以启动开发板，但是从零开始配置确实比较复杂，而且很花时间，这也是一次宝贵的经历。

#29033: RVV interleave/deinterleave 内存访问优化

- PR: <https://github.com/opencv/opencv/pull/29033>
- 创建时间: 2026-05-14
- 合并时间: 2026-05-19
- 状态: 已合并
- 目标分支: 4.x

平台与架构

- 平台: RISC-V
- 架构扩展: RVV 1.0
- HAL: RVV scalable HAL
- 实测硬件: SpacemiT K1, VLEN=128

修改文件

- `modules/core/include/opencv2/core/hal/intrin_rvv_scalable.hpp`

修改前后代码对比

修改前, `v_load_deinterleave` 对多通道数据使用多条 strided load:

```
1 a = __riscv_vlse##width##_v##suffix##m2(ptr, sizeof(_Tp)*3,
    vlanes);
2 b = __riscv_vlse##width##_v##suffix##m2(ptr+1, sizeof(_Tp)*3,
    vlanes);
3 c = __riscv_vlse##width##_v##suffix##m2(ptr+2, sizeof(_Tp)*3,
    vlanes);
```

修改后, 使用 segmented load 一次性读取交错数据:

```
1 vuint##width##m2x3_t seg =
2     __riscv_vlsege##width##_v_u##width##m2x3((const
    uint##width##_t*)ptr, vlanes);
3
4 a =
    v_reinterpret_as_##suffix(v_uint##width##m2x3_u##width##m2(seg, 0));
5 b =
    v_reinterpret_as_##suffix(v_uint##width##m2x3_u##width##m2(seg, 1));
6 c =
    v_reinterpret_as_##suffix(v_uint##width##m2x3_u##width##m2(seg, 2));
```

修改前, `v_store_interleave` 对多通道数据使用多条 strided store:

```
1 __riscv_vsse##width##m2x3(ptr, sizeof(_Tp)*3, a, vlanes);
2 __riscv_vsse##width##m2x3(ptr+1, sizeof(_Tp)*3, b, vlanes);
3 __riscv_vsse##width##m2x3(ptr+2, sizeof(_Tp)*3, c, vlanes);
```

修改后, 使用 segmented store:

```

1  vuint##width##m2x3_t seg;
2  seg = __riscv_vset_v_u##width##m2_u##width##m2x3(seg, 0,
    v_reinterpret_as_u##width(a));
3  seg = __riscv_vset_v_u##width##m2_u##width##m2x3(seg, 1,
    v_reinterpret_as_u##width(b));
4  seg = __riscv_vset_v_u##width##m2_u##width##m2x3(seg, 2,
    v_reinterpret_as_u##width(c));
5  __riscv_vsseg3e##width##_v_u##width##m2x3((uint##width##_t*)ptr,
    seg, vlanes);

```

优化内容

该 PR 将 RVV scalable HAL 中的 `interleave/deinterleave` 操作从 `vlse` / `vsse` 改为 `vlseg` / `vsseg`：

- 原实现：对 2、3、4 通道数据分别做多条 `strided load/store`。
- 新实现：利用 RVV `segmented load/store` 指令，让硬件一次性处理交错内存布局。
- 影响范围：`split`、`merge`、多通道算术、`resize`、`flip` 等依赖多通道 `load/store` 的路径。

性能比较

我在 PR 中给出了 SpacemiT K1 上的性能数据：

测试项	修改前 MEAN MS	修改后 MEAN MS	提升
Resize 640x480 8UC3 Pure	0.42	0.28	+33.33%
Subtract 640x480 8UC3 Convert	1.70	1.36	+20.00%
Resize 640x480 8UC3 Split	1.19	0.99	+16.81%
Add 3840x2160 32FC3	69.51	60.14	+13.48%
Subtract 640x480 8UC3 Pure	0.57	0.50	+12.28%
ScaleAdd 3840x2160 32FC4	97.76	89.24	+8.72%
Absdiff 3840x2160 32FC4	94.64	90.35	+4.53%

单通道对照基本无回归：

测试项	修改前 MEAN MS	修改后 MEAN MS	变化
Subtract 3840x2160 8UC1	5.32	5.32	0.00%
Add 1920x1080 32FC1	5.31	5.32	-0.19%
Multiply 3840x2160 32FC1	20.43	20.24	+0.93%

感想

这个pr就真的是实打实的优化了。经历了前面一系列配置板子的困难，我已经可以成功面对在这里遇到的编译失败或者其他的构建问题了。

我们知道，在riscv有关的优化分为两种：一种是通用内联优化，一种是HAL(Hardware Acceleration Layer)硬件加速层的优化。

关于通用内联，OpenCV 有一套统一的 Universal Intrinsics API（例如 `v_add`, `v_float32x4`）。前端算法开发者只需要用这套 API 写一次代码，编译器在编译时，就会自动将其“翻译”成对应硬件平台的底层向量指令。

OpenCV支持两个版本的RISC-V向量扩展：

1. RVV 0.7.1（固定长度）：通过 `intrin_rvv071.hpp` 实现，提供128位固定长度向量支持。
2. RVV 1.0（可变长度）：通过 `intrin_rvv_scalable.hpp` 实现，支持向量长度无关（VLA, Vector Length Agnostic）的向量操作。

RVV的向量长度在运行时确定，通过 `vTraits` 模板获取，我们可以从 `intrin_rvv_scalable.hpp` 这个文件中找到所有通用内联：

```

1  template <> struct vTraits<vuint8m2_t> {
2      static inline int vlanes() { return __riscv_vsetvlmax_e8m2();
3      }
4      // ...
5  };

```

关于HAL，对于 RISC-V 平台，这种优化的核心依赖于 RISC-V 的“V”扩展（RVV, RISC-V Vector Extension）。HAL通过宏重定向机制实现函数替换，将OpenCV的HAL接口函数重定向到RISC-V特定的实现。先检查有没有HAL实现，如果有相关实现就走针对RISCV的具体实现，如果没有就回退到通用的SIMD或者标量实现。

关于知识这里我就不过多介绍了，下面我讲讲我的pr是怎么优化，怎么验证的，以及遇到的一些问题和解决办法。

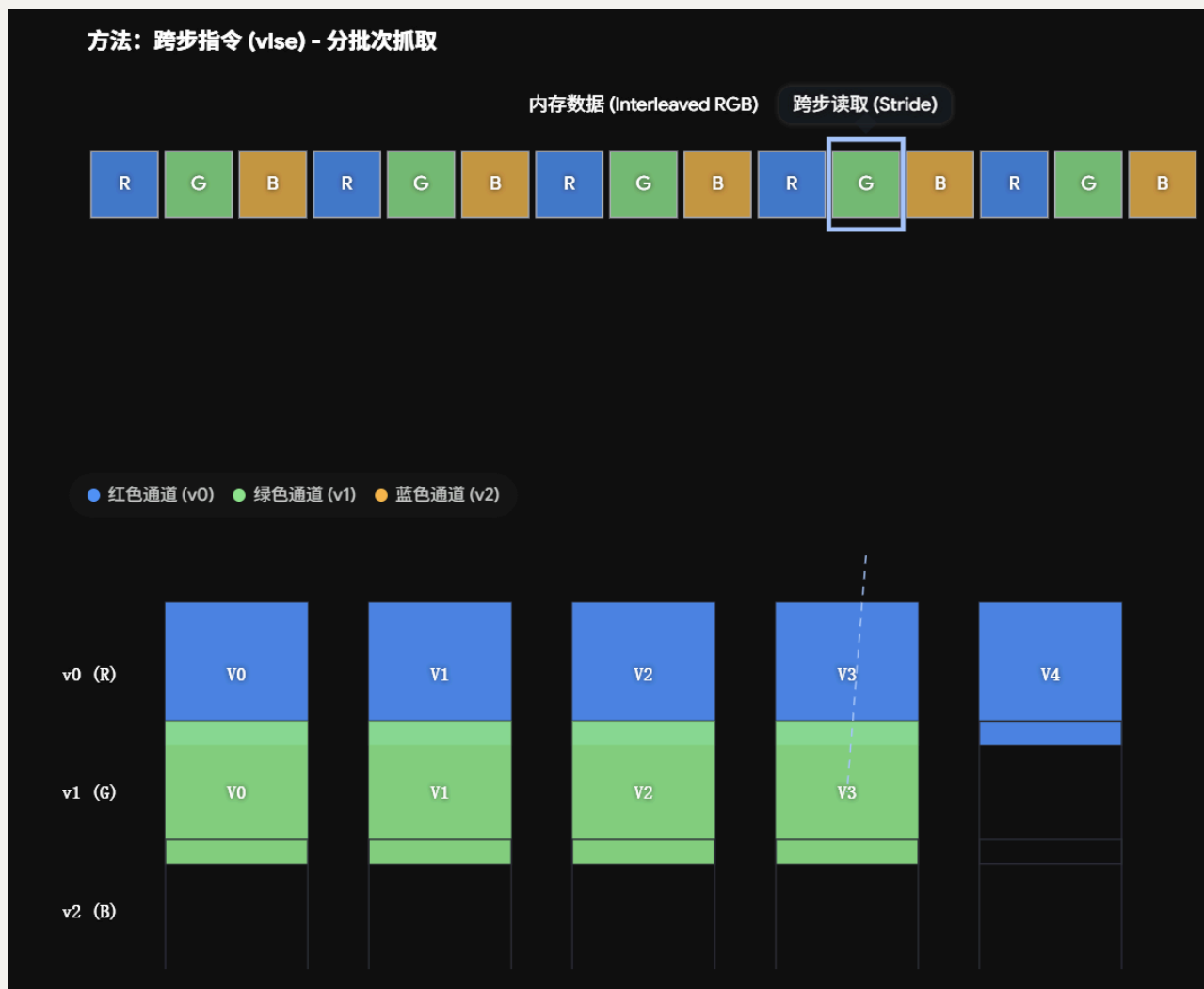
首先，向量计算单元和内存数据的排列方式是不同的。

- 内存里的真实排布 (AoS - Array of Structures): 图像数据是按像素连续存的，比如 `[R, G, B, R, G, B, R, G, B...]`。
- 计算单元的期望排布 (SoA - Structure of Arrays): 向量指令希望一次性处理同类数据，它希望寄存器 1 里全是 `[R, R, R]`，寄存器 2 里全是 `[G, G, G]`。

把内存里的 AoS 转换成计算单元需要的 SoA，这个过程就叫 Deinterleave（解交织）；反过来写回内存叫 Interleave（交织存储）。

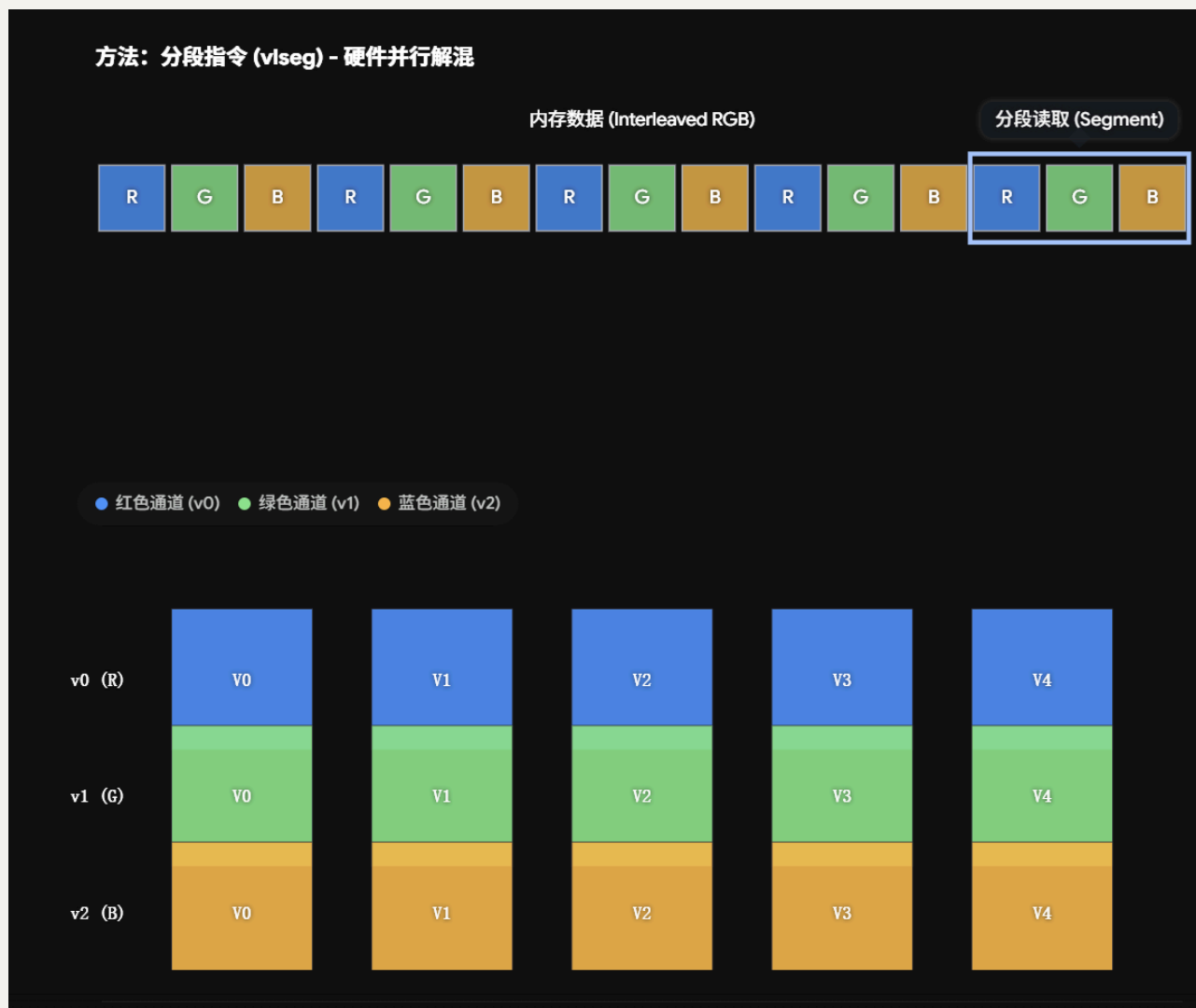
原有的代码为了分离 RGB 三个通道，使用3条独立的指令。

- 指令 1 (`vlse`): 步长为 3。读第 0, 3, 6 个字节（拿走所有的 R）。
- 指令 2 (`vlse`): 步长为 3。读第 1, 4, 7 个字节（拿走所有的 G）。
- 指令 3 (`vlse`): 步长为 3。读第 2, 5, 8 个字节（拿走所有的 B）。



我们知道，CPU 从内存读数据不是一个字节一个字节读的，而是一次读取Cache Line进 L1 缓存。当你执行指令 1 去跳着读 **R** 时，CPU 把包含 RGB 的这 64 字节全拉进来了，但只挑了里面的 R，把 G 和 B 晾在了一边。接着执行指令 2 读 **G**，CPU 可能又要重新寻址。所以性能瓶颈在于跳跃访问+重复执行。

那怎么解决这个问题？好消息，现在的rvv已经有相关指令的支持。现在的代码使用 `vlseg`，向内存一次请求数据。将整块的三通道数据 `[R,G,B,R,G,B...]` 完整拉入 CPU。进入 CPU 后，RISC-V 的向量寄存器文件（VRF）内部的交叉互连网络（Crossbar）会自动进行物理路由，瞬间将 R 分发到 0 号寄存器，G 分发到 1 号寄存器，B 分发到 2 号寄存器。全过程只用了 1 条指令，总结起来就是：一次读取，内部分发。



这是访问时的修改，写回内存也是一样的。

原有的 `vsse` (Vector Strided Store) 跨步存储通过三条指令，将单个向量寄存器中的元素，按指定的字节步长跳跃式地写入内存。

现在使用 `vssegN` (Vector Segment Store) 分段交织存储，使用一条指令，将 N 个连续的向量寄存器中的数据，在硬件层面上自动交织 (Interleave)，然后以连续的内存块形式写入物理内存。

那我们怎么做修改，我找到了docs.riscv.org/reference/vector-c-intrinsics/_attachments/v-intrinsic-spec.pdf。这个是RISC-V Vector C Intrinsic Specification Document。居然足足有4000页！里面有相关指令的用法。在pr的过程中，还遇到了我本地的GCC14和CI的Clang编译器的不同。主要是针对指令的写法不同会有警告，不过在这个文档里都有详细的指令写法，最终也是通过了。

后续就是在板子上做性能测试，先跑基线数据，然后修改代码，重新跑测试，得到了显著的性能提升。这个修改并不是针对某个函数的具体优化，而是关于内存的load和store修改，影响的函数有很多，涉及到交织和解交织的文件函数都有性能提升，比如模块的`cv::resize`, `cv::remap`, `cv::warpAffine`, `cv::warpPerspective`, (`cv::cvtColor`), 和`core`模块的`cv::split`, `cv::merge`等一系列函数，在pr中有具体列出，这里就不再展开。

#29057: RVV `cv_16s` L2 norm 结果错误修复

- PR: <https://github.com/opencv/opencv/pull/29057>
- 创建时间: 2026-05-18
- 合并时间: 2026-05-19
- 状态: 已合并
- 目标分支: `4.x`
- 关联问题: `#29052`

平台与架构

- 平台: RISC-V
- 架构扩展: RVV 1.0
- 实测硬件: Banana Pi BPI-F3 / SpacemiT K1
- CPU: 8 cores, RVV 1.0
- 编译器: GCC 14.2.1
- OpenCV: `4.14.0-pre`
- HAL: RVV HAL enabled

修改文件

- `hal/riscv-rvv/src/core/norm.cpp`
- `modules/core/test/test_arithm.cpp`

修改前后代码对比

修改前，`f64m1` 标量初始化却使用了 `e32m1` 的最大 VL：

```
1  __riscv_vfmv_s_f_f64m1(0, __riscv_vsetvlmax_e32m1())
```

修改后，改为匹配 `f64m1` / 64-bit element width 的 `e64m1`：

```
1  __riscv_vfmv_s_f_f64m1(0, __riscv_vsetvlmax_e64m1())
```

该修改应用于两个路径：

- `NormL2_RVV<short, double>`
- `MaskedNormL2_RVV<short, double>`

新增回归测试：

```
1  TEST(Core_Norm, NORM_L2SQR_16SC4_large)
2  {
3      const int sizes[] = {1, 116, 40};
4      Mat src(3, sizes, CV_16SC4, Scalar::all(16384));
5      const double expected =
6          static_cast<double>(src.total()) * src.channels() *
7          16384.0 * 16384.0;
8      EXPECT_EQ(expected, cv::norm(src, NORM_L2SQR));
9  }
```

修复内容

原问题出现在 RISC-V RVV HAL 的 `cv_16s` L2 / L2SQR norm 累加路径。失败用例为：

```
1  src[0] ~ 16sC4 3-dim (1 x 116 x 40)
2  expected: 3370900308417
3  actual:   -173296
```

根因是 reduction initializer 的 vector length 类型不匹配：向 `f64m1` 写 scalar 初值时使用了 `e32m1` 的最大 VL。该错误会污染最后的 horizontal reduction，导致大规模 `cv_16s` 平方和得到负数或错误值。

验证结果：

```
1  opencv_test_core --  
   gtest_filter='Core_Norm.NORM_L2SQR_16SC4_large'  
2  [ PASSED ] 1 test.  
3  
4  opencv_test_core --  
   gtest_filter='Core_Norm/ElemwiseTest.accuracy/0'  
5  [ PASSED ] 1 test.
```

感想

说实话，这个就是个小错误，偶然刷issues，发现正好是riscv的，看到就顺手修了。然后在 `modules/core/test/test_arithm.cpp` 补充了一个测试。很快就过了。因为这个问题是正确性的错误，也不涉及复杂的调用链。有了前面pr的经历看riscv有关的代码也比较好懂了。

#29058: RVV convertScale 深度转换路径优化

- PR: <https://github.com/opencv/opencv/pull/29058>
- 创建时间: 2026-05-18
- 合并时间: 2026-05-19
- 状态: 已合并
- 目标分支: `4.x`

平台与架构

- 平台: RISC-V
- 架构扩展: RVV 1.0
- 实测硬件: Banana Pi BPI-F3, 8-core RISC-V board
- 编译器: `riscv64-unknown-linux-gnu-g++ 14.2.1`
- OpenCV: `4.14.0-pre`
- 构建: Release, RVV HAL enabled

修改文件

- `hal/riscv-rvv/src/core/convert_scale.cpp`

修改前后代码对比

修改前，RVV HAL 的 `convertScale()` 只覆盖部分路径，例如 `CV_8U -> CV_32F` 和 `CV_32F -> CV_32F`，其他深度转换返回未实现：

```
1 case CV_8U:
2     if (ddepth == CV_32F)
3         return convertScale_8U32F(...);
4     return CV_HAL_ERROR_NOT_IMPLEMENTED;
5
6 case CV_32F:
7     if (ddepth == CV_32F)
8         return convertScale_32F32F(...);
```

修改后新增 3 条 RVV 优化路径：

```
1 case CV_16U:
2     if (ddepth == CV_32F)
3         return convertScale_16U32F(...);
4
5 case CV_16S:
6     if (ddepth == CV_32F)
7         return convertScale_16S32F(...);
8
9 case CV_32F:
10    if (ddepth == CV_8U)
11        return convertScale_32F8U(...);
```

新增 `CV_16U -> CV_32F` 的典型实现：

```
1 auto vec_src = __riscv_vle16_v_u16m4(src_row + j, vl);
2 auto vec_src_f32 = __riscv_vfvcvt_f(vec_src, vl);
3 auto vec_fma = __riscv_vfmadd(vec_src_f32, a, vec_b, vl);
4 __riscv_vse32_v_f32m8(dst_row + j, vec_fma, vl);
```

新增 `CV_32F -> CV_8U` 的典型实现:

```
1 auto vec_src = __riscv_vle32_v_f32m8(src_row + j, vl);
2 auto vec_fma = __riscv_vfmadd(vec_src, a, vec_b, vl);
3 auto vec_dst_u16 = __riscv_vfncvt_xu(vec_fma, vl);
4 auto vec_dst = __riscv_vncclipu(vec_dst_u16, 0, __RISCV_VXRM_RNU,
    vl);
5 __riscv_vse8_v_u8m2(dst_row + j, vec_dst, vl);
```

优化内容

该 PR 扩展 RVV HAL 的 `convertScale` 实现, 为 `Mat::convertTo()` 常见深度转换增加架构专用路径:

- `CV_32F -> CV_8U`
- `CV_16U -> CV_32F`
- `CV_16S -> CV_32F`

优化被限制在 `hal/riscv-rvv/src/core/convert_scale.cpp`, 不会影响非 RISC-V 平台, 也不改变 OpenCV 通用 `convertTo()` 逻辑。

结果

PR 中使用 `opencv_perf_core` 对 `*convertTo*` 进行 benchmark, 强制 20 samples:

```
1 OPTIMIZED_PAIRS: 24 cases, gmean 2.6009x
2
3 CV_32F -> CV_8U:    16.0718x
4 CV_16U -> CV_32F:    1.2334x
5 CV_16S -> CV_32F:    1.1376x
```

额外说明:

- `CV_32F -> CV_8U` 是收益最大的路径。
- 在测试尺寸、通道数和 alpha 组合下, `CV_32F -> CV_8U` 加速范围约为 `14.5x` 到 `18.8x`。

- 未触及路径的第二轮对照结果接近中性： `UNTOUCHED_OTHER: gmean 0.9941x`。

具体实现步骤

这个 PR 的主要工作是在 OpenCV 的 RISC-V RVV HAL 后端中，为 `convertScale` 增加了几条新的向量化类型转换路径。它优化的是 `Mat::convertTo()` 在特定数据深度转换时的底层执行过程，包括 `CV_32F -> CV_8U`、`CV_16U -> CV_32F` 和 `CV_16S -> CV_32F`。也就是说，用户层面的 API 没有变化，`convertTo()` 的数学语义也没有变化；变化发生在更底层的硬件加速层，使这些转换在支持 RVV 的 RISC-V 平台上可以走更快的实现。

我们首先要理解 OpenCV 是怎么做类型转换的。类型转换本质上是对每个像素或每个矩阵元素执行同一个公式：先把源数据乘以 `alpha`，再加上 `beta`，最后转换成目标数据类型。可以理解为 `dst = saturate_cast<dst_type>(src * alpha + beta)`。其中 `saturate_cast` 的作用是保证转换后的值不会溢出。例如从 `CV_32F` 转成 `CV_8U` 时，浮点数会被缩放、偏移、舍入，并限制在 0 到 255 的范围内；而从 `CV_16U` 或 `CV_16S` 转成 `CV_32F` 时，主要是把 16 位整数扩展成 32 位浮点数，再进行乘加运算。

那我们的代码改动就是把这些逐元素操作改写成 RVV 向量指令形式。对于 `CV_16U -> CV_32F` 和 `CV_16S -> CV_32F`，代码先用 RVV 一次加载一批 16 位整数，然后使用 `widening conversion` 把它们扩展成 32 位浮点数，再用向量 `fused multiply-add` 完成 `src * alpha + beta`，最后批量写回目标数组。对于 `CV_32F -> CV_8U`，代码先批量加载 32 位浮点数，做向量乘加，然后把浮点结果转换并窄化成无符号整数，最后再窄化到 8 位并写回。整个过程仍然保持 `convertTo()` 原本的含义，只是把原来一个元素一个元素处理的工作变成了一批元素同时处理。

它之所以比原来快，是因为 RVV 可以利用 SIMD 思想让一条指令处理多个数据元素，减少循环次数、减少逐元素转换的开销，也能更好地利用 RISC-V 向量寄存器。尤其是 `CV_32F -> CV_8U` 这种转换，本身包含浮点乘加、浮点到整数转换、窄化和饱和处理，单个元素的操作比较复杂，因此向量化后的收益非常明显。PR 的 benchmark 中，优化后的转换组合整体几何平均提升约 2.6 倍，其中 `CV_32F -> CV_8U` 的提升最大，达到约 16 倍。

这是一个针对 RISC-V RVV 的特定优化。它不会影响 x86、ARM，也不会改变 OpenCV 通用 `convertTo()` 的代码逻辑。`Mat::convertTo()` 在执行时会先通过 HAL 机制尝试调用平台相关的 `convertScale` 实现；如果当前平台是启用了 RVV HAL 的 RISC-V，并且转换类型正好匹配这三条新增路径，就会进入我们的优化代码。否则，OpenCV 仍然会回退到原来的通用实现。因此，这个 PR 的影响范围是明确且安全的：它只提升 RISC-V RVV 平台上部分常见类型转换的性能，而不改变其他平台和其他转换路径的行为。

调用链可以简单记成：`Mat::convertTo()` 先判断目标类型和缩放参数，然后通过 `CALL_HAL(convertScale, cv_hal_convertScale, ...)` 尝试进入平台 HAL；在 RISC-V RVV 后端里，`convertScale()` 根据 `sdepth` 和 `ddepth` 分发到这几个新增函数。源码中可以看到 `Mat::convertTo()` 会先调用 HAL，若 HAL 未实现再走通用 `getConvertScaleFunc()`。

感想

这个优化是HAL的，像是做了一个小的project3。其实也不难，因为原本已经有很多类型转换的实现，一共8种数据类型，56种转换，除去原有的，剩下的类型转换我选了8个去补充，最后pr写的是最明显的3个。这个属于是增加代码，也通过了正确性测试的验证，性能提升也可见，所以也比较容易过。

总结

总结这个project，从布置到结束，历时一个月，我提了7个pr，6个被合并。我总结了一下，步骤大概是这样的：面对庞大的代码库，我们要阅读源码，理解调用链，精准的定位问题。然后明确问题和性能瓶颈，理解原有的代码逻辑。然后修改代码，最后验证准确性或者性能提升。我们在这里要特别小心，因为我们往往没办法有管理员一眼的完整视野，我们都是针对一个小模块的修改，我们其实并不清楚别人会怎么用它，能做的就是明确调用链，这很关键。

关于AI，如果没有AI，这个project有点无从下手。我在这个project中更多的使用AI，为了定位问题所在的代码，和明确调用链。对于代码的修改，我让ai给出初步意见，但是我会在官方文档中找到对应依据并验证。不能全部的依赖AI。这个project的体验让我能更进一步接触到真实的问题，并在实际过程中解决配置问题，编译问题，性能优化问题。其实我改动的代码并不多，但是每一处改动我都比较小心谨慎，因为每种写法都有具体的特性，我能做到的就是尽可能完整的分析改动带来的影响。

祝愿OpenCV开源社区越来越好。